

Aqualine Water Billing System

MySQL Database Connector Guide

Architecture & Integration Doc
App: Aquiline-Water-Billing
Driver: mysql2/promise

This guide explains how the **Aqualine Water Billing System** connects to, manages, and synchronizes data with a **MySQL database**. It details the connection configuration, pooling mechanism, automatic schema migrations, and real-time data persistence strategy.

1. Configuration & Credentials (.env)

The application uses dotenv to load MySQL credentials from the environment file (.env):

```
MYSQL_HOST=127.0.0.1      # Database server host
MYSQL_PORT=3306           # Port (default 3306)
MYSQL_USER=root           # MySQL username
MYSQL_PASSWORD=           # MySQL user password
MYSQL_DATABASE=my_db      # Target schema/database name
MYSQL_TABLE_PREFIX=awbc_  # Prefix applied to all system tables
# Or via single URI string: MYSQL_URL=mysql://root:password@127.0.0.1:3306/my_db
```

2. Connection Pool Creation (server.js)

The connection is initialized via createMysqlPool() using the high-performance **mysql2/promise** driver. It maintains a pool of up to 5 concurrent connections with automatic connection queuing and UTF-8 encoding support:

```
function createMysqlPool() {
  if (MYSQL_CONNECTION_URL) {
    return mysql.createPool({ uri: MYSQL_CONNECTION_URL, waitForConnections: true, connectionLimit: 5 });
  }
  return mysql.createPool({
    host: MYSQL_HOST, port: MYSQL_PORT, user: MYSQL_USER, password: MYSQL_PASSWORD,
    database: MYSQL_DATABASE, waitForConnections: true, connectionLimit: 5, charset: 'utf8mb4'
  });
}
```

3. Managed Database Tables (Schema Auto-Generation)

On server startup, initializeDatabase() executes ensureMysqlSchema(), automatically creating all required relational tables with proper types, keys, and indexes if they do not exist:

Table Name	Description & Contents	Key Columns
awbc_customers	Customer accounts, phones, auth tokens, session state	id, phone, login_code, login_token
awbc_payments	M-Pesa STK & manual water purchases, receipts, tokens	id, amount, mpesa_receipt, token_code
awbc_settlements	Automated revenue split (Savings 70% / Operations 30%)	id, payment_id, savings_amount
awbc_refunds	Refund requests, dual-step approval, issued adjustments	id, payment_id, amount, status
awbc_ledger_entries	Double-entry accounting journal for all cash flows	id, type, direction, account, amount

awbc_finance_state	System balance states, float thresholds, and policies	id, policy, balances, last_auto_settlement
--------------------	---	--

4. Data Synchronization & Persistence Architecture

The app implements a high-speed **In-Memory Cache + Asynchronous Write-Through** architecture:

- **Fast Reads:** High-frequency read queries are served instantaneously from dbCache (memory).
- **Debounced Writes:** Updates trigger scheduleDbPersist() which debounces writes by 50ms.
- **ACID Transactions:** Writes execute inside MySQL transactions (START TRANSACTION ... COMMIT) to guarantee zero partial state updates or corruption.

5. Automatic Fallback Protection

If MySQL is unreachable (e.g. server down, invalid password, or missing config), the application does **not crash**. It logs a descriptive warning and automatically falls back to local JSON file storage (data/db.json). When MySQL becomes available, data can be migrated seamlessly.

6. Quick Verification & Testing Checklist

Step	Action	Expected Result
1. Check MySQL	Ensure MySQL service is active and database exists (CREATE DATABASE my_db;)	Database is ready to accept connections.
2. Configure .env	Set MYSQL_USER, MYSQL_PASSWORD, MYSQL_DATABASE in .env	App picks up credentials at startup.
3. Start Server	Run npm start	Log: Aqualine server running at http://localhost:3000
4. Check Tables	In MySQL console, run SHOW TABLES;	Lists awbc_customers, awbc_payments...